Practical 1: Singleton Pattern

Part B: Analysis Questions

Q1. How many objects are created?

Answer: Two different CollegeConfigBad objects are created (config1 and config2).

Q2. Why does the comparison return false? (config1 == config2)

Answer: Because both variables refer to different objects stored at different memory locations.

Q3. What problems can occur if configuration objects are created repeatedly?

Answer: It wastes memory, may cause inconsistent configuration values, and makes the system harder to manage.

Q4. Is creating multiple configuration objects necessary in this scenario? Why?

Answer: No. Only one shared configuration object is needed for the whole application.

Q5. Which software quality attributes are affected?

Answer: Memory efficiency, maintainability, consistency, and performance.

Part C: Problems in the Existing Design

1. Multiple configuration objects are created unnecessarily.
2. Memory is wasted.
3. Configuration data may become inconsistent.

Practical 2: Factory method Pattern

Part B – Analysis Questions

Q1. How does the program decide which notification type to send?

The factory checks the notification type (EMAIL, SMS, or PUSH) and creates the corresponding notification object. The object's send() method is then called.

Q2. Why are multiple if-else statements considered a bad design?

Because every new notification type requires modifying the existing code. This makes the program harder to maintain and violates the Open/Closed Principle.

Q3. What problems can occur if notification creation logic is written repeatedly?

- Code duplication
- Difficult maintenance
- More bugs
- Inconsistent implementation
- Hard to update

Q4. Is centralizing object creation necessary in this scenario? Why?

Yes. Centralizing object creation reduces duplicate code, improves maintainability, and makes adding new notification types easier.

Q5. Which software quality attributes are affected?

- Maintainability
- Extensibility
- Reusability
- Readability
- Scalability
- Flexibility

Part C – Problems in Existing Design

1. Multiple if-else statements.
2. Violates Open/Closed Principle.
3. Object creation is not centralized.
4. Difficult to add new notification types.
5. Code duplication.
6. Poor maintainability.

Practical 3: Builder Pattern

Part B: Analysis Questions

Q1. How is the Student object created in the current implementation?

Answer: It is created using a constructor with many parameters.

Q2. Why is using a constructor with many parameters considered a bad design?

Answer: It is difficult to read, understand, and remember the correct parameter order.

Q3. What problems can occur if constructors have many parameters?

Answer: It increases the chance of passing incorrect values and makes the code harder to maintain.

Q4. Is using the Builder Pattern necessary in this scenario? Why?

Answer: Yes. It makes object creation simpler, clearer, and more flexible.

Q5. Which software quality attributes are affected?

Answer: Readability, maintainability, flexibility, and reliability.

Part C: Problems in the Existing Design

1. Constructor has too many parameters.

2. Easy to pass parameters in the wrong order.

3. Code is difficult to read and maintain.